

Statistics Guide — algorithms and methods behind Tab 3

This document explains **how** each column in the Tab 3 “Generate Statistics” CSV is computed — the specific algorithm, library, and formula behind it — not just what the number means. For a plain-English, biology-oriented explanation of each column instead, see [GUIDE.md, Section 11](#). This document is the companion reference for anyone extending `_statistics.py`, auditing a result, or writing up the method (e.g. a Methods section).

All of this lives in one module, `napari_zf_microglia_ai/_statistics.py`, entered through a single function, `compute_stats()`.

Table of Contents

1. [Pipeline architecture](#)
 2. [Phase 1 — Batch regionprops](#)
 3. [Derived shape metrics from the inertia tensor](#)
 4. [Phase 2a — Surface area \(marching cubes\) and sphericity](#)
 5. [Phase 2a — Skeleton and branching statistics](#)
 6. [Phase 2b — Intensity statistics \(optional\)](#)
 7. [Post-assembly — Spatial statistics \(incl. 7.5 is volume outlier\)](#)
 8. [Post-assembly — Brain region assignment \(optional\)](#)
 9. [Morphotype classification](#)
 10. [Natural-language description generation](#)
 11. [Full column → algorithm cross-reference](#)
 12. [References](#)
-

1. Pipeline architecture

`compute_stats(labels, scale_zyx, image=None, region_lines=None, region_names=None, backend_config=None)` runs in three phases, each chosen to keep the expensive parts vectorised or parallelised rather than looping over labels one at a time in pure Python:

Phase	What	How
1	Batch regionprops (volume, centroid, bbox, inertia tensor, solidity, extent) for every label in one call	cuCIM (GPU) if available, else scikit-image (CPU)
2a	Per-label marching-cubes surface area + skeleton/branch analysis	ThreadPoolExecutor, one worker per label, cropped to that label's bounding box
2b	Per-label intensity statistics (<i>only if an Image layer was supplied</i>)	Same thread-pool pattern as 2a
3	Assemble the DataFrame, then compute statistics that need <i>all</i> labels at once (nearest-neighbour, brain region), then generate descriptions	Vectorised NumPy/SciPy, then one description call per row

Why cropping to the bounding box matters (2a/2b): marching cubes and skeletonization are run on a small 3D crop (`labels[z0:z1, y0:y1, x0:x1] == 1b1`), not the full volume — this is what makes per-label operations on a multi-megavoxel image tractable at all. The crop bounds come directly from Phase 1's regionprops bbox output, so Phase 1 must run first.

GPU backend detection (`_detect_stats_backend()`): tries importing `cupy + cucim.skimage.measure`, then runs a 4×4×4 smoke-test `regionprops_table` call to confirm the CUDA JIT actually compiles and runs (not just that the import succeeded) before committing to the GPU path for the whole run. Falls back to CPU

(`skimage.measure.regionprops_table`) on any failure — including cuCIM being unavailable on Windows, since it has no native Windows wheels (see README/GUIDE’s Windows install notes).

Thread count: `_N_THREADS = max(1, os.cpu_count() // 2)` — half the logical cores, leaving headroom for the GPU-bound phase and the main thread rather than saturating every core.

2. Phase 1 — Batch regionprops

`_batch_regionprops(labels)` calls `regionprops_table` (cuCIM or `skimage` — same API, same property names) once, requesting:

```
_RPROPS = [  
    "label", "area", "centroid", "bbox",  
    "inertia_tensor", "inertia_tensor_eigvals",  
    "axis_major_length", "axis_minor_length",  
    "solidity", "extent",  
]
```

This single call is where most of the raw geometry comes from:

Column(s)	Source property	Notes
volume_vox	area	scikit-image calls the 3D voxel count “area” for historical (2D-first) API reasons
centroid_z/y/x_vox, centroid_z/y/x_um	centroid	voxel centroid × <code>scale_zyx</code> for the μm columns
bbox_z0/y0/x0_vox (start, inclusive), bbox_z1/y1/x1_vox (end, exclusive)	bbox	standard NumPy half-open slicing convention — <code>labels[z0:z1, y0:y1, x0:x1]</code> is exactly the label’s bounding crop
bbox_dz/dy/dx_um	bbox (end – start) × <code>scale_zyx</code>	physical size of the bounding box, not the label’s own extent (a diagonal or branched cell can have a much larger

Column(s)	Source property	Notes
		bbox than its actual volume would suggest — see extent below)
solidity	solidity directly	volume / convex_hull_volume — the library computes the 3D convex hull internally and divides
extent	extent directly	volume / bounding_box_volume
axis_major_length, axis_minor_length	used below	pixel-space ellipsoid-fit lengths, not yet in μm — see §3 for why these need care under anisotropic voxels
$\text{volume_um}^3 = \text{volume_vox} \times \text{sz} \times \text{sy} \times \text{sx}$ (vox_vol = the physical volume of one voxel, computed once from scale_zyx).		

3. Derived shape metrics from the inertia tensor

This is the part of the pipeline most worth reading carefully if you're citing these numbers, because it involves a real approximation under anisotropic voxel scaling ($Z = 1.0 \mu\text{m}/\text{vox}$ vs. $Y = X = 0.174 \mu\text{m}/\text{vox}$ in this project's typical data).

3.1 principal_axis_dir — which axis is the cell elongated along

The 3×3 inertia tensor for every label (from Phase 1) is stacked into an $(N, 3, 3)$ array and eigendecomposed in one batched call, `np.linalg.eigh(IT)`. `eigh` returns eigenvalues in **ascending** order — physically, the eigenvector paired with the **smallest** eigenvalue of the

inertia tensor is the object's axis of elongation (mass concentrated close to that axis contributes little to inertia about it; mass spread far from an axis contributes a lot). So:

```
longest_vecs = eigvecs[:, :, 0] # smallest-eigenvalue
    eigenvector = major axis direction
axis_dirs = ["Z", "Y", "X"][argmax(|component|)] # per label,
    whichever axis the eigenvector points closest to
```

principal_axis_dir is therefore the axis (Z, Y, or X) that the true 3D major axis most nearly aligns with — not a continuous angle, just the dominant of the three.

3.2 axis1_um, axis2_um, axis3_um — the anisotropy caveat

axis_major_length/axis_minor_length from regionprops are computed in **voxel units**, from an ellipsoid fit that implicitly assumes isotropic voxels. Converting to μm correctly requires knowing *which physical axis* the length actually runs along — a length “along Z” and a length “along X” need different scale factors (1.0 vs. 0.174 $\mu\text{m}/\text{vox}$ here) to become physically correct.

The code handles this only for the *major* axis, by construction:

```
axis1_scales = [axis_scale_map[d] for d in axis_dirs]
    # pick sz, sy, or sx per-label, matching
    principal_axis_dir
a1_um = axis_major_length * axis1_scales
a3_um = axis_minor_length * scale_mean #
    scale_mean = (sz+sy+sx)/3 — an approximation
a2_um = (a1_um + a3_um) / 2.0 # NOT
    independently measured — see below
```

Three things worth knowing if you rely on these numbers:

- **axis1_um is scaled correctly** (per-label, using whichever physical axis it actually points along).
- **axis3_um uses the mean of the three axis scales**, not the true axis it points along, because regionprops' minor-axis direction isn't tracked per-label the way the major axis is here. This is an approximation, more accurate the closer $sz/sy/sx$ are to each other (it is *not* a negligible approximation for this project's typical ~5.7:1 Z:XY anisotropy on an axis pointing mostly along Z or mostly along XY).
- **axis2_um is not measured at all** — it's simply $(\text{axis1_um} + \text{axis3_um}) / 2$, a placeholder for the true middle principal axis (which would require a second eigenvector-aligned length

measurement that regionprops doesn't provide directly). Treat axis2_um as illustrative, not a real geometric measurement.

3.3 elongation, eq_diam_um

- elongation = axis1_um / axis3_um (guarded against divide-by-zero with a 1e-10 floor). 1.0 = sphere; higher = more elongated.
 - eq_diam_um = $(6 \cdot \text{volume_um}^3 / \pi)^{1/3}$ — the diameter of a sphere with the same volume as the label. Shape-independent; two cells with identical volume have the same eq_diam_um regardless of how elongated or branched either one is.
-

4. Phase 2a — Surface area (marching cubes) and sphericity

_surface_area(binary, scale_zyx):

1. The label's cropped binary mask is **padded by 1 voxel on every side** (np.pad(binary, 1)) before meshing. This is necessary because marching cubes needs a zero-boundary to close the mesh — without padding, a label touching its own crop's edge would produce an open (non-manifold) surface and an undercounted area.
2. skimage.measure.marching_cubes(padded, level=0.5, spacing=scale_zyx) extracts an isosurface at the 0.5 threshold (the boundary between labeled and unlabeled voxels), directly in physical units via the spacing parameter (so no separate unit conversion is needed afterward).
3. skimage.measure.mesh_surface_area(verts, faces) sums the area of every triangle in the resulting mesh.
4. Returns 0.0 on any failure (e.g. a 1-voxel object with no meaningful surface) rather than raising, so one bad label doesn't abort the whole run.

Sphericity (computed in the main assembly loop, not inside _surface_area):

$$\text{sphericity} = \pi^{1/3} \cdot (6V)^{2/3} / A \quad (V = \text{volume_um}^3, A = \text{surface_area_um}^2)$$

This is the classical Wadell (1935) sphericity — the surface area a perfect sphere of the same volume *would* have, divided by the actual surface area. It's clamped to a maximum of 1.0 (mesh discretisation

noise can occasionally push the raw ratio fractionally above 1.0 for near-spherical labels; a physical sphericity cannot exceed 1.0 by definition).

`surface_to_volume_ratio = surface_area_um2 / volume_um3` — a simpler, non-normalised alternative to sphericity; unlike sphericity it keeps growing without bound as branching increases, rather than saturating.

5. Phase 2a — Skeleton and branching statistics

`_skeleton_stats(binary, scale_zyx)`, using `skimage.morphology.skeletonize` + the `skan` package:

1. `skeletonize(binary)` — thins the 3D binary mask to a topological skeleton (1-voxel-wide medial curve), preserving connectivity and branch structure.
2. `skan.Skeleton(skeleton, spacing=scale_zyx, source_image=binary)` builds a graph representation, with edges already in physical units via `spacing`.
3. `skan.summarize(sk, separator="-")` produces one row per skeleton **branch** (an edge between two graph nodes — a branch point or an endpoint), with columns including `branch-type`, `euclidean-distance`, and `branch-distance` (path length along the branch).

From that branch table:

Column	Computation
<code>n_branches</code>	<code>len(branch_table)</code> — one row per branch
<code>n_endpoints</code>	count of rows where <code>branch-type == 1</code> (skan's code for a branch ending in a free tip, as opposed to a branch connecting two junctions)
<code>mean_branch_len_um</code>	mean of <code>euclidean-distance</code> (straight-line branch endpoint-to-endpoint distance, not path length) across all branches
<code>max_branch_len_um</code>	

Column	Computation
	max of the same euclidean-distance column
branch_tortuosity	mean of branch-distance / euclidean-distance (path length ÷ straight-line distance) over branches with nonzero euclidean-distance; 1.0 = perfectly straight
branch_density	$n_branches / volume_um3 \times 10^6$ (branches per million μm^3 , i.e. per $\sim 100 \times 100 \times 100 \mu m$ cube — the $\times 10^6$ scaling exists purely to keep the numbers in a readable range)
endpoint_density	same normalisation, for $n_endpoints$
process_complexity	$n_endpoints \times mean_branch_len_um / volume_um3$ — a single combined index: more endpoints <i>and</i> longer branches <i>and</i> smaller cell body all push this up

A real gotcha, already found and fixed in this project once (see the branch-radius calibration work): `skan`'s `source_image=` parameter does **not** feed `summarize()`'s per-branch intensity/"mean-pixel-value" columns the way its name suggests — that requires baking the values into the skeleton array itself before constructing `skan.Skeleton`, not passing them as `source_image`. This module doesn't currently use any `source_image`-derived column (only geometry columns), so it isn't affected, but it's a real API trap worth knowing if this module is ever extended to pull intensity-along-skeleton statistics.

Failure mode: if `skan` isn't installed, or skeletonization produces an empty skeleton (can happen for a 1-2 voxel label), `_skeleton_stats` returns `(0, 0, 0.0, 0.0, 1.0)` rather than raising — all branch-derived columns read as zero/neutral for that label instead of aborting the batch.

6. Phase 2b — Intensity statistics (optional)

Only computed when `compute_stats()` is called with an image array (Tab 3's Image-layer selector). `_intensity_stats_worker` crops both the label mask and the source image to the label's bounding box, then:

```
mean_intensity      = mean(image[mask])
integrated_intensity = sum(image[mask])
intensity_cv        = std(image[mask]) / mean(image[mask])
(0.0 if mean == 0)
```

`intensity_cv` (coefficient of variation) is scale-independent — it measures relative spread of brightness within the cell, not absolute brightness, so it's comparable across cells or samples with different overall exposure/gain.

7. Post-assembly — Spatial statistics

`_spatial_stats(centroids_zyx_um, label_ids)` runs once on **all** centroids together (not per-label), using `scipy.spatial.cKDTree` for efficient nearest-neighbour queries — an $O(N \log N)$ k-d tree build rather than an $O(N^2)$ all-pairs distance matrix.

7.1 Nearest neighbours

```
tree = cKDTree(centroids)
dists, idxs = tree.query(centroids, k=3)  # self + 1st NN + 2nd
                                         NN
```

Column 0 of the result is always the point itself (distance 0), so the 1st nearest neighbour is column 1 and the 2nd is column 2.

`nearest_neighbor_label/nearest_neighbor_2_label` map the neighbour's *tree index* back to its actual label ID (tree order and label order aren't guaranteed to match). With exactly 2 cells total, `k` is clamped to 2 and the “2nd nearest neighbour” columns fall back to duplicating the 1st (there is no second neighbour to find).

7.2 Clark-Evans 3D index (`nearest_neighbor_ratio`)

The classical Clark & Evans (1954) index compares an observed mean nearest-neighbour distance to the value expected under **complete spatial randomness** (a homogeneous 3D Poisson point process) at the

same point density. This module computes a **per-cell** version — each cell’s own NND divided by the population’s expected NND — rather than the traditional single population-level summary statistic.

Derivation of the expected 3D nearest-neighbour distance, since this is worth having written down once: for a 3D Poisson process at intensity (density) ρ , the probability that no other point lies within radius r of a given point is the Poisson void probability for a sphere of volume $(4/3)\pi r^3$:

$$P(\text{NND} > r) = \exp(-\rho \cdot (4/3)\pi r^3)$$

The expectation of a nonnegative random variable is $E[X] = \int_0^\infty P(X > r) \, dr$, so:

$$E[\text{NND}] = \int_0^\infty \exp(-\rho \cdot (4/3)\pi \cdot r^3) \, dr$$

Substituting $u = r^3$ and using the standard Gamma-function integral $\int_0^\infty \exp(-a \cdot r^3) \, dr = (1/3) \cdot \Gamma(1/3) \cdot a^{(-1/3)}$, with $a = \rho(4/3)\pi$, and the identity $\Gamma(4/3) = (1/3)\Gamma(1/3)$:

$$E[\text{NND}] = \Gamma(4/3) \cdot (3 / (4\pi \cdot \rho))^{(1/3)}$$

— exactly what the code computes: `math.gamma(4/3) * (3.0 / (4.0*math.pi*density))**(1.0/3.0)`.

`nearest_neighbor_ratio = observed_NND / E[NND]`. Values **< 1** indicate the cell is closer to its neighbour than random chance would predict (local clustering); **> 1** indicates more spread out than random (regularity/dispersion).

A real approximation to know about: `density = N / bbox_volume`, where `bbox_volume` is the axis-aligned bounding box of the *centroid point cloud itself* (max - min per axis, multiplied together) — not the true tissue/brain volume the cells actually live in. If cells are sparse near the edges of their own bounding box (very likely for a roughly ellipsoidal brain region), this systematically **overestimates** density and therefore **underestimates** the expected NND, biasing `nearest_neighbor_ratio` slightly upward (toward “more regular than it really is”) near the population edges. Fine as a same-sample relative measure; treat with more caution when comparing the raw ratio across samples with different cell-count-derived bounding boxes.

7.3 local_density_100um

`tree.query_ball_point(centroids, r=100.0, return_length=True) - 1`
— count of other centroids within a 100 μm radius sphere of each cell (the -1 removes the point counting itself). A direct local crowding count, independent of the Clark-Evans model assumptions above.

7.4 depth_normalized

`clip(centroid_z_um / max(centroid_z_um), 0, 1)` across the current label set — 0 = the shallowest (lowest Z) cell in this result set, 1 = the deepest. Recomputed per-run, so it is **not** comparable across different fish/samples unless they share the same Z range by construction — it's a within-sample relative depth, not an absolute one.

7.5 is_volume_outlier (not computed by this module)

Unlike every other column on this page, `is_volume_outlier` is not produced inside `compute_stats()` at all — it's appended afterward, in `_widget.py`'s `_on_generate_stats()`, once the `DataFrame` comes back. It exists to hand the two “is this cell real?” edge cases GT sweeps and pipeline stages can't resolve on their own back to a human:

- **Too big** — could be two touching cells the pipeline's merge/split logic left joined instead of separating.
- **Too small** — could be genuine debris, or could be a real but unusually small microglia; below the deletion threshold, both routes already remove it outright (see [GUIDE.md §6a, Final min-size fraction](#) — Cellpose-SAM's final safety-net stage, the Pixel Classifier's own volume filter cutoff), but the gray zone between that threshold and the confirmed floor survives untouched on purpose on both routes.

```
too_big    = df["volume_vox"] > max_ceiling if max_ceiling is not
           None else False
too_small  = df["volume_vox"] < min_floor   if min_floor   is not
           None else False
df["is_volume_outlier"] = too_big | too_small
```

`max_ceiling` (`max_volume_recommended_vox` in config) and `min_floor` (`min_volume_recommended_vox`, the same field Common Settings' Min volume slider reads) are both cross-fish histories tracked the same way as every Tab 5 sweep's recommendations — see `_update_gt_history()` in `_widget.py`. `min_floor` is a never-rising floor (`mode="min"`); `max_ceiling` is its never-falling mirror (`mode="max"`, the same direction

used for `branch_radius`). Both only move when Tab 3's "**This is verified ground truth**" checkbox is ticked for the run that measured them — an unverified/uncorrected prediction can widen neither bound, though every run, verified or not, is still flagged against whichever bounds were last confirmed.

8. Post-assembly — Brain region assignment (optional)

Only computed when Tab 3 is given a Shapes layer of boundary lines/polylines (drawn by the user, one curve per boundary between two anatomical regions) and a list of region names.

8.1 Geometric primitive: point-to-polyline projection

`_polyline_side_and_dist(cy, cx, pts)` finds, for a 2D point and a multi-segment polyline, which *segment* is nearest and which *side* of that segment the point falls on:

1. For every consecutive vertex pair (*a*, *b*) in the polyline, project the point onto the **infinite line** through *a*→*b*, then clamp the projection parameter *t* to $[0, 1]$ — this restricts the projection to the actual segment, not the infinite line (standard point-to-segment-distance technique).
2. Track the minimum distance across all segments — that segment is the “nearest segment.”
3. At that nearest segment, compute the 2D cross product $dx_seg \cdot (cy - ay) - dy_seg \cdot (cx - ax)$. Its **sign** tells you which side of the segment's direction the point is on: this project's fish are oriented head (small X) → tail (large X), with boundary curves drawn top→bottom (increasing Y) by convention (see GUIDE.md §7a), so a negative cross product means the point is posterior to (right of) that boundary.

8.2 Region index from multiple boundaries

`_assign_brain_regions` sorts all supplied boundary curves by their **mean X coordinate** (ascending = most anterior first) — this makes boundary order well-defined regardless of the order the user drew them in. For each cell centroid, it counts how many boundaries the cell is posterior to:

```

idx = 0
for boundary in boundaries_sorted_anterior_to_posterior:
    if point_is_posterior_to(boundary):
        idx += 1
region = region_names[idx]

```

With k boundary curves this naturally produces $k+1$ regions (e.g. 1 boundary $\rightarrow$ anterior/posterior; 2 boundaries $\rightarrow$ 3 regions). `region_boundary_dist_um` is simply the minimum distance (in μm) from the cell's centroid to *any* boundary curve — cells near this value are close to a region edge and might reasonably be considered ambiguous/borderline.

9. Morphotype classification

`_classify_morphotype(sphericity, solidity, elongation, n_branches, surface_to_volume_ratio)` — a hand-tuned rule-based decision list, evaluated in priority order (first matching rule wins):

```

if elongation > 3.5 and n_branches <= 3:
    return "Rod-shaped"
if sphericity > 0.70 and solidity > 0.80 and n_branches <= 2:
    return "Amoeboid"
if n_branches >= 6 and sav > 2.0 and sphericity < 0.55:
    return "Ramified"
if n_branches >= 4 and sphericity < 0.65:
    return "Intermediate-ramified"
else:
    return "Intermediate"

```

This is **not** a fitted/learned classifier — it's a fixed set of thresholds chosen to match the standard microglia morphological categories used in the field (ramified/resting vs. amoeboid/activated, plus rod-shaped and intermediate states), evaluated directly on this module's own geometric outputs. Because it's rule-based and fully deterministic, it's reproducible and auditable, but it hasn't been validated against an independent ground-truth morphotype labeling — treat it as a descriptive heuristic, not a diagnostic classification, unless/until it's been checked against expert-annotated examples.

10. Natural-language description generation

The description column is produced by one of four interchangeable backends, selected via `backend_config["backend"]`:

Backend	Method	Requires
rule (default)	<code>_rule_based_description()</code> — a template that stitches together shape/surface/branch phrases from fixed thresholds on the already-computed columns (sphericity, solidity, elongation, branch/endpoint counts, morphotype)	nothing — fully offline, deterministic
ollama	POSTs a formatted prompt to a local Ollama server's <code>/api/generate</code> endpoint	a running local Ollama install, no API key
openai	POSTs to <code>{api_url}/v1/chat/completions</code> (OpenAI-compatible chat endpoint)	an API key
claude	POSTs to <code>https://api.anthropic.com/v1/messages</code>	an API key

The three API backends all format the same shared prompt template (`_STATS_PROMPT`) with that row's already-computed statistics (volume, elongation, dominant axis, sphericity, solidity, branch/endpoint counts, mean branch length, morphotype, centroid) and ask for **one sentence, max 40 words**. Every backend call is wrapped in a try/except that returns an inline `"[Backend error: ...]"` string on failure (timeout, bad key, unreachable server) rather than raising — one row's description failing doesn't abort the whole statistics run.

The rule-based backend is the only one exercised by anything else in the pipeline (it has no external dependency), and is what runs by default with no configuration.

11. Full column → algorithm cross-reference

Column	Section
label, volume_vox, volume_um3, centroid_*, bbox_*	§2
solidity, extent	§2
principal_axis_dir	§3.1
axis1_um, axis2_um, axis3_um	§3.2
elongation, eq_diam_um	§3.3
surface_area_um2, sphericity, surface_to_volume_ratio	§4
n_branches, n_endpoints, mean_branch_len_um, max_branch_len_um, branch_tortuosity, branch_density, endpoint_density, process_complexity	§5
mean_intensity, integrated_intensity, intensity_cv	§6
nearest_neighbor_label, nearest_neighbor_dist_um, nearest_neighbor_2_label, nearest_neighbor_2_dist_um	§7.1
nearest_neighbor_ratio	§7.2
local_density_100um	§7.3
depth_normalized	§7.4
is_volume_outlier	§7.5 — computed in <code>_widget.py</code> , not <code>_statistics.py</code>
brain_region, region_boundary_dist_um	§8
morphotype	§9
description	§10

For the plain-English meaning of every column (rather than the algorithm behind it), see [GUIDE.md §11](#).

12. References

- Wadell, H. (1935). *Volume, Shape, and Roundness of Quartz Particles*. Journal of Geology, 43(3) — the sphericity formula used in §4.
- Lorensen, W. E., & Cline, H. E. (1987). *Marching Cubes: A High Resolution 3D Surface Construction Algorithm*. ACM SIGGRAPH Computer Graphics, 21(4) — the surface-meshing algorithm behind `surface_area_um2` (§4).
- Clark, P. J., & Evans, F. C. (1954). *Distance to Nearest Neighbor as a Measure of Spatial Relationships in Populations*. Ecology, 35(4) — the spatial-randomness index generalised to 3D in §7.2.
- Nunez-Iglesias, J., et al. — `skan`: skeleton analysis in Python, used throughout §5.
- van der Walt, S., et al. (2014). *scikit-image: image processing in Python*. PeerJ 2:e453 — `regionprops_table`, `marching_cubes`, `skeletonize` (§2, §4, §5).
- Virtanen, P., et al. (2020). *SciPy 1.0*. Nature Methods — `cKDTree` (§7).